



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра математического обеспечения и стандартизации информационных
технологий (МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №2
по дисциплине
«Распределенные СУБД»

Выполнил студент группы ИКБО-66-23

Смирнов А.Ю.

Принял ст. преподаватель

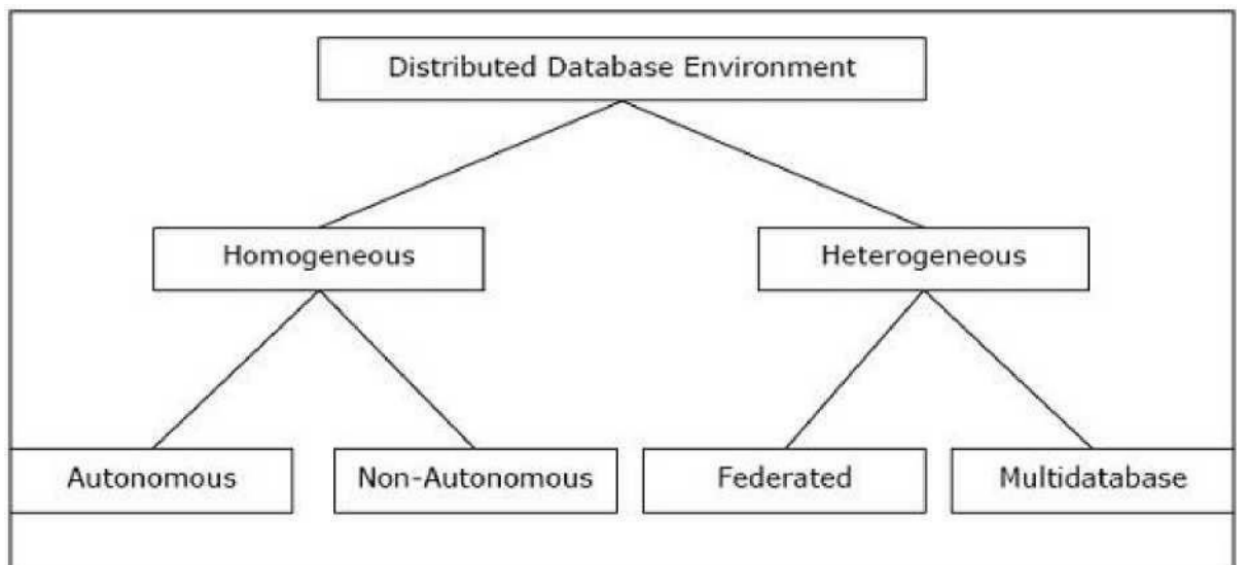
Гуличева А.А.

Москва 2026

Теоретическое введение

Типы распределённых систем

Распределенные базы данных можно широко классифицировать на однородные и гетерогенные среды распределенных баз данных, каждая из которых имеет дополнительные подразделения, как показано на следующем рисунке:



Однородные распределенные базы данных

Разберём, как работают однородные РБД на примере сайтов (подразумевается некое веб-приложение, установленное на различных узлах). В однородной распределенной базе данных все сайты используют идентичные СУБД и операционные системы. Её свойства:

- на сайтах используется очень похожее программное обеспечение;
- сайты используют идентичные СУБД или СУБД одного и того же производителя;
- каждый сайт знает обо всех других сайтах и взаимодействует с другими сайтами для обработки пользовательских запросов;
- доступ к базе данных осуществляется через единый интерфейс, как если бы это была одна база данных.

Типы однородной распределенной базы данных

Существует два типа однородной распределенной базы данных:

- автономный – каждая база данных независима и функционирует самостоятельно. Они интегрированы управляющим приложением и используют передачу сообщений для обмена обновлениями данных;
- неавтономный – данные распределяются по однородным узлам, а центральная или главная СУБД координирует обновления данных по сайтам;

Гетерогенные распределенные базы данных

В гетерогенной распределенной базе данных разные сайты имеют разные операционные системы, продукты СУБД и модели данных. Его свойства:

- различные сайты используют разные схемы и программное обеспечение;
- система может состоять из множества СУБД, таких как реляционная, сетевая, иерархическая или объектно-ориентированная;
- обработка запросов является сложной из-за разнородных схем;
- обработка транзакций является сложной из-за различий в программном обеспечении;
- сайт может не знать о других сайтах, поэтому сотрудничество при обработке пользовательских запросов ограничено.

Типы гетерогенных распределенных баз данных

- федеративные – гетерогенные системы баз данных независимы по своей природе и объединены вместе, так что они функционируют как единая система баз данных;
- без федерации – в системах баз данных используется центральный координационный модуль, через который осуществляется доступ к базам данных.

Практическая часть:

```
C:\Users\Redmi>docker run --name cas1 -p 9042:9042 -d -m 2g -e MAX_HEAP_SIZE="1G" -e HEAP_NEW_SIZE="256M" -e CASSANDRA_CLUSTER_NAME=MyCluster -e CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch -e CASSANDRA_DC=datacenter1 cassandra:latest
45245a689efd09e47090ebb74d0f7266eb04676d97639491f402eca412192cf1
```

Рисунок 1 – Запуск контейнер с первым узлом

```
C:\Users\Redmi>docker run --name cas2 -d -m 2g -e MAX_HEAP_SIZE="1G" -e HEAP_NEW_SIZE="256M" -e CASSANDRA_SEEDS="172.17.0.2" -e CASSANDRA_CLUSTER_NAME=MyCluster -e CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch -e CASSANDRA_DC=datacenter1 cassandra:latest
f1821952018dbc33bfd2644204ffffdcfcd066cee6636b98318ee0c784d9b117f
```

Рисунок 2 – Присоединение к первому узлу следующего узла

```
C:\Users\Redmi>docker run --name cas3 -d -m 2g -e MAX_HEAP_SIZE="1G" -e HEAP_NEW_SIZE="256M" -e CASSANDRA_SEEDS="172.17.0.3" -e CASSANDRA_CLUSTER_NAME=MyCluster -e CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch -e CASSANDRA_DC=datacenter2 cassandra:latest
e35591ba855c43b4054f99cc7e68f6f2e0f31eddf4d968ea6d421fd245ca22bb
```

Рисунок 3 – Создание узла в «удалённом» датацентре для образования кластера

```
C:\Users\Redmi>docker exec -ti cas1 nodetool status
Datacenter: datacenter1
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
-- Address      Load          Tokens   Owns (effective)  Host ID                               Rack
UN 172.17.0.2    119.82 KiB    16       100.0%            c95a201a-fd9d-4152-8517-a1866e7034e3  rack1
UN 172.17.0.3    85.1 KiB     16       100.0%            1482af4d-e110-4de1-bf58-630d7eb7c76b  rack1

Datacenter: datacenter2
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
-- Address      Load          Tokens   Owns (effective)  Host ID                               Rack
UJ 172.17.0.4    30.9 KiB     16       ?                  d3f0ef84-1166-4ea7-82b7-eceae98b3efc  rack1
```

Рисунок 4 – Проверка работоспособности кластера

```
C:\Users\Redmi>docker exec -it cas1 cqlsh
Connected to MyCluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.6 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh> CREATE KEYSPACE mykeyspace WITH replication = {
```

Рисунок 5 - Переход в утилиту cqlsh для выполнения скрипта

```
cqlsh> CREATE KEYSPACE mykeyspace WITH replication = {
... 'class' : 'NetworkTopologyStrategy',
... 'datacenter1': 1,
... 'datacenter2': 1
... };
cqlsh> USE mykeyspace;
```

Рисунок 6 – Создание пространства keyspace

```
cqlsh> USE mykeyspace;
cqlsh:mykeyspace> |
```

Рисунок 7 – Выполнение скрипта для использования keyspace

```
cqlsh> USE mykeyspace;
cqlsh:mykeyspace> CREATE TABLE Student (
... id int primary key,
... name text,
... lastname text,
... direction text,
... course int,
... birth_day text
... );
cqlsh:mykeyspace>
```

Рисунок 8 – Создание таблицы

```
cqlsh:mykeyspace> INSERT INTO Student (id, name, lastname, direction, course, birth_day)
... VALUES(1, 'Smirnov', 'Andrey', 'IT', 3, '17.01.2005');
cqlsh:mykeyspace> SELECT * FROM mykeyspace.Student;
```

id	birth_day	course	direction	lastname	name
1	17.01.2005	3	IT	Andrey	Smirnov

(1 rows)
cqlsh:mykeyspace> |

Рисунок 9 – Заполнение ранее созданной таблицы и её вывод

```
C:\Users\Redmi>docker exec -ti cas2 cqlsh
Connected to MyCluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.6 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh> SELECT * FROM mykeyspace.Student;
```

id	birth_day	course	direction	lastname	name
1	17.01.2005	3	IT	Andrey	Smirnov

(1 rows)
cqlsh> |

Рисунок 10 – Вывод таблицы со второго узла

Вывод

В ходе выполнения данной практической работы были изучены архитектурные особенности и принципы функционирования распределенной СУБД Apache Cassandra:

- был развернут кластер из трех узлов в среде Docker, распределенных по двум виртуальным дата-центрам (datacenter1 и datacenter2);
- отработаны навыки работы с языком запросов CQL, включая создание пространства ключей (keyspace) с настройкой стратегии репликации NetworkTopologyStrategy;
- на практике подтверждена децентрализованная природа системы: данные, созданные на одном узле (cas1), успешно синхронизировались и стали доступны для чтения на другом узле (cas2), что демонстрирует механизм автоматической репликации.

Ответы на контрольные вопросы

1. Виды согласованности (Consistency)

В Cassandra согласованность – не фиксированная настройка, а регулируемый параметр.

- Строгая согласованность (Strong Consistency): гарантирует, что после завершения обновления любой последующий запрос на чтение вернет обновленное значение.
- Согласованность в конечном счете (Eventual Consistency): Система гарантирует что, если новые обновления не будут поступать, через некоторое время все реплики станут идентичными. Это позволяет Cassandra работать с минимальными задержками, так как клиенту не нужно ждать подтверждения от всех узлов.

2. CAP-теорема: Теория и реальность

Теорема Эрика Брюера утверждает, что распределенная система может одновременно обладать только двумя из трех свойств:

1. C (Consistency) – согласованность. Все узлы видят одни и те же данные одновременно.
2. A (Availability) – доступность. Каждый запрос получает успешный отклик.
3. P (Partition Tolerance) – устойчивость к разделению. Система продолжает работать, даже если связь между узлами потеряна.

3. Распределенность и децентрализация

Cassandra спроектирована как система Leaderless architecture, что отличает её от многих традиционных СУБД.

- Распределенность – данные автоматически распределяются по нескольким узлам (нодам) в кластере. Это позволяет масштабировать систему горизонтально, просто добавляя новые серверы.

- Децентрализация – в Cassandra нет «главного» узла (Master/Primary). Все узлы равноправны. Любой узел может принять любой запрос от клиента.